

Analysis & Reflection

Level 1 reflection

For Level 1, I created a simple page with a header, footer, and main content area. I made it twice: one with a flexbox layout and another with a grid layout. I only changed the link in the `<head>` section of my HTML file to see one or another version. Both pages look the same on desktop, tablet, and mobile devices.

For flexbox, I used `flex-direction: column` to make a column out of the body and `flex-grow: 1` for the main section to make it grow in case there is some extra space. In case of the Grid version, I used one property - `grid-template-rows: 70px 1fr 60px` to define the heights of the header, main section, and footer.

There were fewer lines of code for the Grid version, as one property replaced a few ones that I had to use for Flexbox. However, this difference is minor on this simple layout, yet I expect the difference to become more noticeable with more complicated ones. I found Flexbox easier to understand than Grid at the beginning since I thought about Flexbox as boxes that are put into one column. While in the case of Grid, I needed to think about what I was going to do before writing a single line.

The responsive feature is also the same in both layouts since I did not need to change anything except the size of the header, since everything was already stacked, and I just needed to write a media query at 600px to make it smaller on phones.

In conclusion, I will say that I think that I will use Flexbox for cases when I need a column layout as a navigation bar, buttons row or just center something in a box. I will use Grid in case when I need to design a whole page that requires a layout with rows and columns like in the next Levels.

Level 2 reflection

For level 2, I built a responsive restaurant called Umutuzo Kitchen. The website is designed to work well on mobile phones, tablets, and desktop screens. I included the main information a customer may need on the page, such as the restaurant introduction, navigation links, meals, today's special, opening hours, and a short message about visiting the restaurant. I wanted the page to look simple but still feel like an actual restaurant website with information someone could actually want.

For the Grid version, I used CSS Grid to create the 70/30 layout using `grid-template-columns: 7fr 3fr`. I also used Grid for the menu items and changed the layout to one column on smaller screens. Both versions use the same HTML structure, but the layout is achieved differently.

Between the two, I would personally choose Flexbox because I found the code easier to understand and demonstrate. I also learned that Flexbox is useful when working with one row or one column, while Grid is more useful when working with rows and columns together. However, I understand that both have different strengths and can be useful depending on the layout I am trying to create.

One of the main things I learned from this activity is that every line of CSS can contribute to the final result. A small change in padding, width, flex-basis, or grid columns can affect how the whole page looks. I also learned that leaving too many things at their default values can sometimes cause unexpected results, so it is important to set the properties I need instead of always depending on the browser defaults only.

In general, this activity helped me understand the practical difference between Flexbox and Grid and gave me more confidence in creating responsive layouts.

Level 3 reflection

I designed a more advanced responsive website with a header, navigation bar, left sidebar, main content area, right sidebar, and footer. I intended to create a website template that proved effective for all screen sizes and felt like one coherent page rather than a number of disjointed parts put together.

In the Flexbox variant, I used nested Flexbox containers to create the three-column structure. Overall, the left sidebar takes up around 20% of the space, the main content area uses 60%, and the right sidebar takes the remaining 20%. I also applied different Flexbox properties, for example, 'flex-basis' and 'order', to determine the positioning. For smaller screen sizes, I

modified the layout so that the main content part appeared first, and the sidebars were moved below it.

For the Grid version of the website, I used CSS Grid to create the same layout and named the fundamental areas of the grid. This was beneficial, as I was able to specify the positions of the header, menu, sidebars, content areas, and footer. I also applied media queries to set the format of the grid for tablet and mobile devices. This helped me realise how Grid simplifies page design tasks.

Between the two options, Flexbox is more familiar to me since I have implemented it in simple layouts. However, Grid is more convenient to use in three-column designs that contain many blocks. I have discovered that Flexbox is particularly useful when it comes to arranging particular elements, while Grid serves the purpose of describing the overall layout of the page.

One of the things I learned from Level 3 was about responsive design and the complexities that it presents. Responsive design is not just about finding ways to narrow the widths of all the sections; rather, I must decide how exactly the sections relocate and which order the sections will appear in on small screens. Through the course, I learned different concepts of responsive design such as flex-basis, grid-template-columns, order, and grid-template-areas and their roles.

Overall, with the knowledge I gained from Level 3, I became more skilled and confident in creating complicated designs and understanding how a webpage functions across devices.

Level 4 reflection

For Level 4 we built the same Holy Grail layout twice, once with Flexbox and once with CSS Grid, and the two approaches felt different from the start.

Which was easier to implement?

Flexbox was quicker to get something on screen. A flex column for the page, a flex row for the sidebars and main content, and ``margin-left: auto`` to push the user menu right all came together fast. Grid took a bit more planning, because we had to decide on rows and columns first. After that it was easier to maintain. Collapsing a sidebar meant changing one column width, while in Flexbox we also had to manage ``flex-basis`` and ``overflow``.

Which required less code?

Neither was dramatically shorter. Grid needed less code for the card reflow, where ``repeat(auto-fill, minmax(250px, 1fr))`` replaced several flex properties. Flexbox needed more workarounds, such as ``order`` to move the main content above the sidebars on tablet and mobile. In Grid we just rearranged ``grid-template-areas``.

Which was more intuitive?

Flexbox felt more natural for one-dimensional things like the navigation bar and the header, where items sit in a single line. Grid felt more natural for the page as a whole, because we could see the layout in the code. The footer showed this best: with subgrid, the headings and link lists line up across all three columns without any extra effort.

What differences did we notice?

The two versions don't look quite identical. When the last row of cards is short, Flexbox stretches the cards to fill the row, while Grid keeps them all the same width. Getting Flexbox to match would need extra work, and this was the clearest example for us of the two systems thinking differently. Flexbox sizes items based on their content, and Grid sizes them based on the grid.

When would we use each one?

We would use Flexbox for components: navigation menus, toolbars, buttons, and anything that sits in a row or column. We would use Grid for page structure and for anything that needs rows and columns to line up, like card galleries and footers. In practice they work best together. Our Grid page still uses flexible alignment inside the smaller components, and the best layouts probably mix both.

Overall, Level 4 taught us that this isn't really a choice between one or the other. Flexbox is better at distributing items along one direction, and Grid is better at defining the whole structure. Knowing which question we're asking (content first or layout first) tells us which tool to use.